

STRATIV

AI Receipt Scanner

Developer Guide

Audience Developers joining or maintaining the codebase

App version 1.0.0 · Expo SDK 54 · React Native 0.81.5

Platform Android (internal distribution; iOS builds configured but out of scope)

Document date 20 August 2026

Source [docs/developer-guide.html](#) — edit the HTML, regenerate the PDF

1. What this app is

An internal Strativ tool for office expense processing. Staff photograph paper receipts with their phone; the app reads each one with a vision model, stores the result on the device, and at the end of the month produces one Excel **Bill Approval Sheet** that a manager signs.

Three properties shape almost every decision in the codebase:

- **Local-first.** SQLite is the source of truth. Staff scan all month, often with no signal. Nothing in the scanning path may require the network except the one AI call.
- **Monthly, not real-time.** The deliverable is a document per month, not a live dashboard.
- **Read by non-developers.** The exported sheet is the product for everyone outside the scanning team. Formatting is a feature, not polish.

Explicit non-goals

- **No Google Sheets integration.** Writing to an organisation sheet needs OAuth the app never had. It was built, then deliberately removed. Do not reintroduce it.
- **No app store release.** Internal distribution only.
- **No user accounts.** There is no login anywhere in the app.
- **No CSV export.** Removed on purpose — see §9.

2. Running it locally

Prerequisites: Node 20+, pnpm, an Android device or emulator, and an EAS account with access to the project.

```
pnpm install
cp .env.example .env          # then fill in the keys below
pnpm start                    # add --clear if the bundler caches a stale env
```

EXPO GO WILL NOT RUN THIS APP

It uses native modules (SQLite, camera, updates) that Expo Go does not bundle. You need a development build: `npm run development-builds`, install the resulting APK once, then `pnpm start` connects to it.

Environment variables

Variable	Required	Purpose
EXPO_PUBLIC_OPENROUTER_API_KEY	Yes	The AI call. Without it every scan fails.
EXPO_PUBLIC_SUPABASE_URL	No	Shared team database. Omit and the app runs fully local.
EXPO_PUBLIC_SUPABASE_ANON_KEY	With URL	Required whenever the URL is set, or every sync is rejected.

ON THE EXPO_PUBLIC_PREFIX

These values are compiled into the JavaScript bundle. Anyone with the APK can extract them. That is accepted for an internal build — use a spend-capped OpenRouter key — but it is why the Supabase anon key holds no delete permission anywhere. See the System Documentation for the full threat model.

Everyday commands

```
pnpm start                # dev server
pnpm test                 # jest-expo, 96 tests
npx tsc --noEmit          # typecheck
npx expo lint             # eslint
npx expo install <pkg>    # install with an SDK-compatible version
```

3. Repository map

Path	What lives there
app/	Expo Router screens. Five tabs plus a receipt detail route.
services/	All logic with a side effect: AI, SQLite, export, sync, config.
contexts/	React state providers. <code>receipts-context</code> is the single source of truth.
components/	Presentational components. <code>components/ui/</code> holds primitives.
constants/	<code>theme.ts</code> (all design tokens), <code>receipt-ui.ts</code> (type icons, labels, colours).
utils/	Pure helpers: <code>format.ts</code> , <code>image-quality.ts</code> .
hooks/	Colour scheme, theme colour, OTA updates.
types/receipt.ts	The domain types. Read this file first.
supabase/schema.sql	Re-runnable DDL and RLS policies for the shared database.
__mocks__/	<code>expo-sqlite</code> mapped onto Node's <code>node:sqlite</code> so storage tests run real SQL.
.eas/workflows/	EAS workflow definitions used by the <code>npm run build</code> scripts.

Screens

Route	Role
app/(tabs)/index.tsx	Dashboard. Current-month card, stats, category breakdown, recent receipts.
app/(tabs)/history.tsx	List with type filter chips and debounced merchant search.
app/(tabs)/capture.tsx	Camera and the processing state machine. Centre tab, raised orange button.
app/(tabs)/export.tsx	Month cards to the export config modal.
app/(tabs)/settings.tsx	Office name, sheet columns, data actions, OTA status.
app/receipt/[id].tsx	Detail: photo viewer, edit, delete, single-receipt export.
app/_layout.tsx	Root providers and the OTA check on start.

Tab order is deliberate: Home, History, **Scan**, Export, Settings. Scan sits in the middle as a raised circle because it is the action staff perform dozens of times a day.

4. The data flow

```
camera / gallery
|
v
validateImage      exists, <= 20 MB, <= 33 MP
|
v
assessReceiptImage on-device quality gate, no network, no AI
|                  blurry / dark / not-a-receipt -> reject
|                  (user may override: "Process Anyway")
v
parseReceiptWithRetry OpenRouter -> Gemini 2.5 Flash, JSON mode
|                    retry with backoff, best-result tracking
v
normalizeResponse   ISO dates, Bangla numerals, confidence recalibration
|
v
ReceiptsProvider.addReceipt
|
+--> SQLite (source of truth)
|
+--> remote-db: photo to Storage bucket, row to Postgres
      (fire-and-forget; failure is retried, never surfaced as a failed scan)
|
v
export: monthly Bill Approval Sheet (XLSX), or raw JSON
```

The quality gate matters commercially: it runs on a 160 px thumbnail in a few milliseconds and costs nothing, so a blurry photo is rejected before it can spend an API call. It is a heuristic, not a classifier, which is why the override button must stay.

5. Module reference

services/ai-vision.ts

The AI layer. It exports exactly two functions — `validateImage` and `parseReceiptWithRetry` — and everything else is internal on purpose.

- **Endpoint:** OpenRouter chat completions, model in the single `MODEL_ID` constant (`google/gemini-2.5-flash`), JSON mode, temperature 0, base64 data URI.
- **Prompt:** tuned for Bangla and Bengali handwriting, including a Bangla alphabet reference and numeral-conversion rules. Retry attempts 2+ switch to a harder second-pass prompt.
- **Retry:** the image is processed once and reused; backoff is 2 s then 4 s. The best-scoring attempt is kept if none clears the acceptance gate (`total` present, calibrated confidence ≥ 0.45).
- **401 and 402 abort immediately** — a bad key or exhausted credits cannot be fixed by retrying.
- **It never throws.** A failure returns a populated error object so the capture screen always has something to render.

services/storage.ts

SQLite CRUD, stats, and the sync bookkeeping. Three tables:

Table	Contents
<code>receipts</code>	One row per scan. <code>items</code> is JSON text. Indexed on <code>created_at</code> , <code>invoice_type</code> , <code>merchant_name</code> , and <code>substr(receipt_date,1,7)</code> for month queries.
<code>config</code>	Key/value app settings — office name, optional endpoint.
<code>export_columns</code>	Which sheet columns are on, and their order.

Two behaviours to respect when editing this file:

- The schema uses `CREATE TABLE IF NOT EXISTS` , so a new column needs a **guarded `ALTER TABLE`** — check `PRAGMA table_info` and tolerate a duplicate-column error. See MAINTENANCE.md §5.
- Initialisation is shared through a single `initPromise` . Two concurrent `initDatabase()` calls on a cold database previously ran the migration twice and produced `duplicate column name: synced_at` .

services/xlsx-export.ts

The only document format in the app. `exportReceiptsAsSheet()` produces the same layout for one receipt or a filtered list, so there is one sheet to learn. The monthly Bill Approval Sheet has a merged office-name heading, columns driven by the Settings config, money as numeric cells formatted `#,##0.00` so Excel can total them, a summary block, and four signature blocks spaced proportionally across the sheet width.

USE XLSX-JS-STYLE, NEVER PLAIN XLSX

The community SheetJS build silently discards cell styles on write — no error, just an unstyled sheet. Related: `XLSX.read` drops styles even with `cellStyles: true`, which is why `buildBillApprovalSheet` is extracted as a pure function and tested directly rather than through a written file.

services/remote-db.ts

The optional shared database, spoken to over PostgREST with plain `fetch`. Do **not** add `@supabase/supabase-js` — its auth, realtime and storage bundle buys nothing over one POST with two headers, on a platform where bundle size is shipped to every phone.

- **Two-way.** Each device pushes its own scans and pulls the whole office's, so one phone shows another phone's receipts and the monthly sheet covers the team.
- **Ownership rule** in `importRemoteReceipts()`: a row with a `file://` image path was scanned here and is never overwritten by the pull. Rows that came from the database are refreshed, so a correction reaches everyone.
- **Photos** go to a private Storage bucket, resized first. The row carries the object path; the pull turns it back into a URL.
- **Never delete local rows to match the remote.** An empty read is indistinguishable from a blocked or failed one, so "the table looks empty, clear the phone" would destroy data — including scans not yet uploaded. The pull only ever adds or refreshes.
- Raw `postgresql://` and `mysql://` URLs are rejected with an explanation: React Native has no TCP socket.

services/config.ts

Validation over the config tables, plus the three sync entry points: `syncPendingReceipts()` (offline catch-up on app start), `pullRemoteReceipts()`, and `reuploadAllReceipts()` (Settings → Data, used to recover after the remote table is truncated).

services/export.ts

Raw JSON export and `shareFile`. That is all it does now.

6. State management

`ReceiptsProvider` (mounted in the root layout) owns database initialisation, the receipt list, recent receipts, stats, and filter/search state. The rules are short and worth keeping:

- Every mutation goes through `addReceipt`, `updateReceiptById` or `removeReceipt`. Screens consume `useReceipts()`.
- No screen calls `initDatabase` or fetches a list directly.
- `status === 'initializing'` drives first-load skeletons; background refreshes keep the previous list on screen rather than flashing a spinner.

`contexts/export-context.tsx` holds export screen state only — periods, selected month, generating, share. It is deliberately separate: export state is short-lived and irrelevant to every other screen.

7. Design system

The theme follows the official Strativ design system, and these are hard rules rather than preferences:

- **One accent:** Strativ Orange `#FE5001`. No purple, no indigo.
- **No gradients anywhere.** Solid fills only.
- **"Black" is Warm Black** `#1A0E1C`, never `#000`. Dark mode uses the neutral dark scale (`#0B0E13` family), not warm black.
- **No emoji in product UI.**
- The Strativ logo stays visible in the dashboard header lockup.
- Category colours come from the data-viz palette in `constants/receipt-ui.ts` — never brand orange.

Practical conventions

- **No hardcoded hex in screens.** Every colour is a token in `constants/theme.ts`, in light/dark pairs. Need a new colour? Add a token.
- **Two fonts:** Expletus Sans for headings (`Type.display`), Inter for everything else. Use the `Type` families rather than `fontWeight` — Android does not synthesise weights for custom fonts, so a bold that works on iOS silently renders regular on Android.
- **Icons** go through `components/ui/icon-symbol.tsx`, which maps SF Symbol names to MaterialIcons behind a strict `IconSymbolName` type. A missing icon fails typecheck instead of rendering "?" on Android.
- **Safe areas** come from `useSafeAreaInsets()`. Never hardcode top padding.
- Kebab-case file names, function components, TypeScript throughout.

8. Testing

```
pnpm test
```

96 tests across three suites, on the `jest-expo` preset. The interesting part of the setup is `__mocks__/expo-sqlite.js`, which maps `expo-sqlite` onto Node's built-in `node:sqlite`. Storage tests therefore execute real SQL — migrations, constraints and the ownership rule are genuinely exercised rather than asserted against a stub.

Suite	Covers
<code>xlsx-export.test.ts</code>	Sheet layout: merges, column spans, signature block spacing, numeric money cells, Bangla round-trip.
<code>remote-db.test.ts</code>	Wire format: endpoint paths, auth headers, upsert preference, row mapping, photo upload, ownership rule.
<code>storage.config.test.ts</code>	Config and export-column storage, month queries, sync bookkeeping.

What is *not* covered: no component or navigation tests, and no test hits OpenRouter or Supabase for real. Both are deliberate — the value has been in the sheet layout and the sync rules, which is where the bugs were.

9. Traps that have already cost time

Each of these was a real defect. They are listed because the fix is invisible in the code that now looks obvious.

Bangla text arrived as mojibake

The old CSV export produced `à|«à§ ...` instead of Bangla merchant names: a CSV carries no encoding declaration, so spreadsheets guess the code page. XLSX stores UTF-8 inside the file. CSV was deleted rather than patched with a BOM, because the flat `=== RECEIPT SUMMARY ===` layout was also unreadable for the people approving bills. **Do not reintroduce it.**

Cell styles vanished on write

Plain `xlsx` discards styles silently. The dependency is `xlsx-js-style` and must stay.

duplicate column name: synced_at

A `Promise.all` triggered two concurrent `initDatabase()` calls, running the migration twice. Fixed with a shared `initPromise` plus tolerance for duplicate-column errors.

eas update failed on a WebAssembly file

`expo export` included the web platform, which pulls `wa-sqlite.wasm`. Fixed by pinning `"platforms": ["android", "ios"]` in `app.json`. Web is not a target.

EAS install failed with ERR_PNPM_IGNORED_BUILDS

`pnpm 11` no longer reads the `pnpm` field in `package.json`, and `strictDepBuilds` now defaults to `true`. The fix lives in `pnpm-workspace.yaml`:

```
allowBuilds:
  unrs-resolver: true
```

The ownership rule is not "empty image_uri"

It used to be, before photos synced. Now imported rows hold a bucket URL, so the test is a `file://` prefix. Reverting it would make a scan whose photo failed to upload look like someone else's row — and get it clobbered on the next pull.

10. Common changes

Swap the AI model

Change the single `MODEL_ID` constant in `services/ai-vision.ts`. The model must support image input and JSON mode. Verify against a Bangla handwritten receipt before shipping — that is where models differ most.

Add a column to the exported sheet

Add an entry to `DEFAULT_COLUMNS` in `services/config.ts`, then handle the field in `formatReceiptRow` in `services/xlsx-export.ts`. Existing installs keep their stored column config, so a new column is off until the user enables it in Settings.

Add a column to the database

Guarded `ALTER TABLE` in `services/storage.ts`, mirrored in `supabase/schema.sql` with `add column if not exists`. Apply the SQL **before** shipping the app update — PostgreSQL rejects a payload naming an unknown column, which would break sync entirely rather than partially.

Add an icon

Add the SF Symbol name and its MaterialIcons equivalent to `MAPPING` in `components/ui/icon-symbol.tsx`. Typecheck enforces this.

Ship a change

JavaScript-only changes go over the air:

```
eas update --channel main --environment preview -p android -m "..."
```

A new native module, config plugin, or app.json native change needs a new build instead. Details in [MAINTENANCE.md §1](#) and [§2](#).

11. Where the documentation lives

Document	Purpose
docs/developer-guide.html	This document. How the code works and how to change it.
docs/system-documentation.html	Architecture, data model, security, environments, operations.
MAINTENANCE.md	Living runbook: exact commands for releases, OTA, key rotation, schema changes, release notes.
CLAUDE.md	Conventions and hard rules, loaded automatically by AI agents working in the repo.
AGENTS.md	Expo-specific agent guidance: documentation links, EAS workflows.
RELEASE_NOTES.docx	One living release document, newest block at the top. Source: RELEASE_NOTES.html.
.kiro/specs/	The original monthly-export specification, kept for the reasoning behind the pivot away from Google Sheets.

The two PDFs are generated — edit the HTML and run `docs/build-pdf.sh`. MAINTENANCE.md and CLAUDE.md stay markdown because they are edited constantly and diffs matter more than typography.